

Lecture 07

Linalg-on-Tensors

linalg.generic MatMul/Conv 표현과 NPU compiler lowering 관점

Linalg-on-Tensors in an AI/NPU Compiler Pipeline

Keep structured tensor semantics long enough to tile, fuse, vectorize, and map to NPU kernels.



오늘의 위치

StableHLO/TOSA 이후 , Bufferization 이전의 핵심 중간 단계

1-6 강 : MLIR IR 기본 구조 , dialect, tooling, StableHLO/TOSA 입력 IR

7 강 : high-level tensor op 를 structured compute IR 인 Linalg 로 표현

8 강 이후 : SCF/Affine loop, pass, rewrite, bufferization, vector, custom NPU dialect 로 확장

Learning Objectives

Linalg-on-Tensors 가 NPU compiler 에서 왜 중요한지 설명한다 .

`linalg.generic` 의 네 가지 핵심 필드를 읽는다 .

MatMul 을 indexing_maps 와 iterator_types 로 직접 표현한다 .

Conv2D NHWC/HWCF 를 loop domain 과 access map 으로 표현한다 .

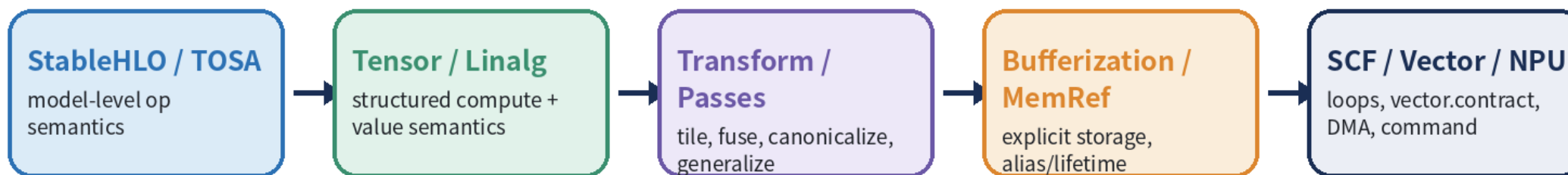
tile footprint, accumulator pressure, epilogue fusion 후보를 도출한다 .

Linalg-on-Tensors 위치

Structured compute IR before bufferization and hardware lowering

Linalg-on-Tensors in an AI/NPU Compiler Pipeline

Keep structured tensor semantics long enough to tile, fuse, vectorize, and map to NPU kernels.



NPU compiler decisions enabled by Linalg

- indexing_maps expose data access functions for tile footprint analysis
- iterator_types distinguish parallel loops from reduction loops
- region body keeps scalar compute explicit for fusion and vectorization
- tensor outputs delay physical memory layout until bufferization

왜 Linalg 가 NPU 에 중요한가 ?

Op 이름만으로는 SRAM footprint, halo, accumulator lifetime 을 알기 어렵다 .

Linalg 는 loop/data mapping 을 affine map 과 iterator 로 명시한다 .

Tensor result 는 physical storage 결정을 늦추고 , fusion/tiling 여지를 유지한다 .

NPU compiler 는 Linalg 단계에서 PE mapping 과 memory traffic 을 추정할 수 있다 .

Tensor value semantics

Tensor result 는 값처럼 취급된다 : mutation 을 직접 노출하지 않는다 .

`outs` operand 는 결과 shape 와 destination contract 를 제공한다 .

Bufferization 단계에서 실제 memref allocation, alias, in-place update 를 결정한다 .

NPU backend 는 layout/memory space 결정을 너무 일찍 고정하지 않는 것이 중요하다 .

Destination style example

```
%init = tensor.empty() : tensor<8x32xf32>
%zero = linalg.fill ins(%c0 : f32)
                                outs(%init : tensor<8x32xf32>) -> tensor<8x32xf32>
%C = linalg.matmul ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
                                outs(%zero : tensor<8x32xf32>) -> tensor<8x32xf32>
```

Named op vs linalg.generic

Named op: ``linalg.matmul``, ``linalg.conv_2d_nhwc_hwcf`` 처럼 common pattern 을 짧게 표현한다 .

Generic op: indexing map, iterator type, scalar payload 가 모두 드러난다 .

NPU compiler 는 named op 로 빠르게 pattern match 하고 , generic 으로 일반화해 schedule analysis 를 수행할 수 있다 .

너무 빨리 loop 로 낮추면 fusion/vectorization 기회가 사라진다 .

linalg.generic Anatomy

indexing_maps + iterator_types + ins/outs + region body

Anatomy of linalg.generic

A structured operation = iteration space + access maps + iterator semantics + scalar payload region.

```
#matmul = {  
  indexing_maps = [  
    affine_map<(m,n,k) -> (m,k)>, // A  
    affine_map<(m,n,k) -> (k,n)>, // B  
    affine_map<(m,n,k) -> (m,n)>   // C  
  ],  
  iterator_types = ["parallel", "parallel", "reduction"]  
}  
%C = linalg.generic #matmul  
  ins(%A, %B : tensor<MxKxf32>, tensor<KxNxf32>)  
  outs(%init : tensor<MxNxf32>) {  
    ^bb0(%a: f32, %b: f32, %c: f32):  
      %p = arith.mulf %a, %b : f32  
      %s = arith.addf %c, %p : f32  
      linalg.yield %s : f32  
  } -> tensor<MxNxf32>
```

indexing_maps

Loop IVs -> tensor indices; key for tile footprint and layout analysis.

iterator_types

Parallel vs reduction dimensions; legal schedule and accumulator behavior.

ins / outs

Operands define iteration domain; outs carry destination/result shape.

region body

Scalar payload: elementwise math, reduction update, yield result.

linalg.generic Field Checklist

필드	의미	NPU 질문
indexing_maps	Loop IV -> operand index	tile footprint 는 ?
iterator_types	parallel/reduction semantics	PE mapping 과 accumulator 는 ?
ins/outs	input/result shape contract	output tile 과 destination 은 ?
region body	scalar compute payload	epilogue fusion 가능한가 ?

indexing_maps 직관

MatMul: $A(m,k), B(k,n), C(m,n)$

Conv: $\text{input}(n, \text{oh}+kh, \text{ow}+kw, c), \text{filter}(kh, kw, c, f), \text{output}(n, \text{oh}, \text{ow}, f)$

Affine map 은 tile 이 읽고 쓰는 tensor subset 을 계산하는 근거다 .

Stride/dilation/padding 이 들어오면 input map 과 boundary 처리 policy 가 중요해진다 .

iterator_types 직관

parallel: 서로 다른 output element 또는 independent update

reduction: 같은 output element 에 누적되는 dimension

NPU 에서 reduction loop 는 accumulator lifetime 과 partial sum storage 를 결정한다 .

잘못된 iterator classification 은 race, atomic 필요 , 잘못된 fusion legality 로 이어진다 .

Named MatMul / Conv 예제

```
func.func @named_ops(%A: tensor<8x16xf32>,
                    %B: tensor<16x32xf32>,
                    %input: tensor<1x30x30x16xf32>,
                    %filter: tensor<3x3x16x32xf32>)
  -> (tensor<8x32xf32>, tensor<1x28x28x32xf32>) {
    %c0 = arith.constant 0.0 : f32
    %mm_init = tensor.empty() : tensor<8x32xf32>
    %mm_zero = linalg.fill ins(%c0 : f32)
                        outs(%mm_init : tensor<8x32xf32>) -> tensor<8x32xf32>
    %mm = linalg.matmul ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
                        outs(%mm_zero : tensor<8x32xf32>) -> tensor<8x32xf32>

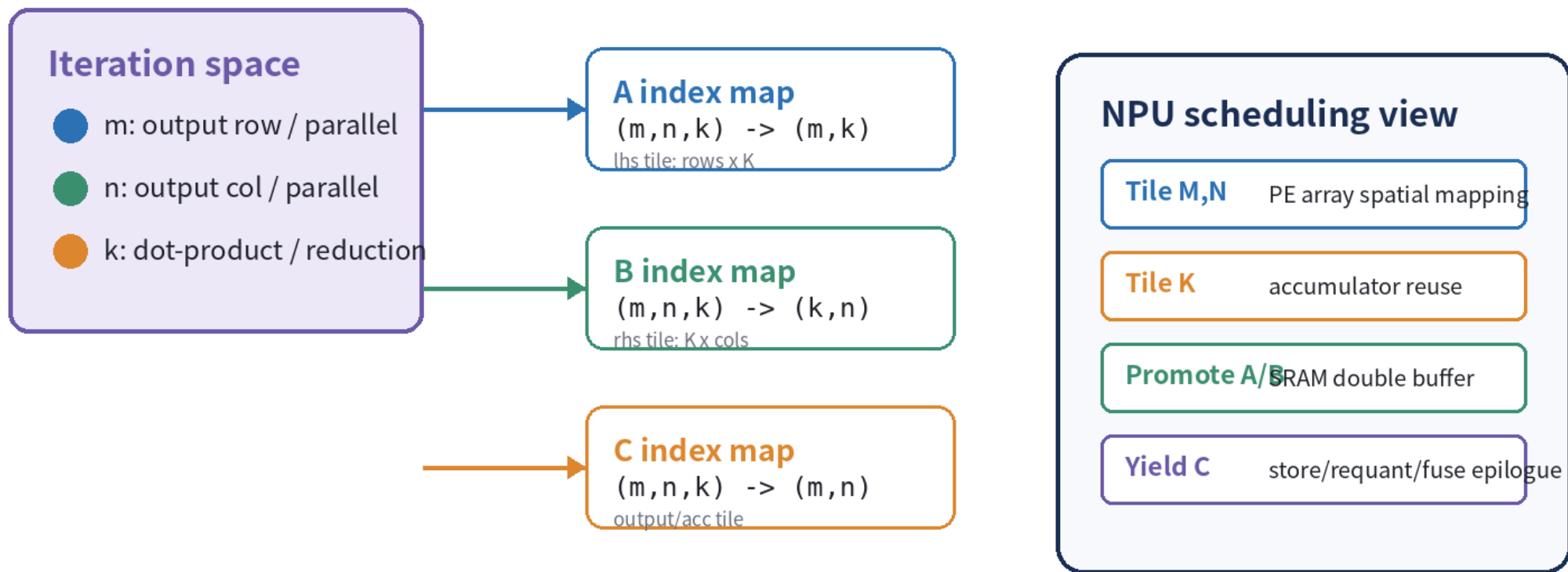
    %cv_init = tensor.empty() : tensor<1x28x28x32xf32>
    %cv_zero = linalg.fill ins(%c0 : f32)
                        outs(%cv_init : tensor<1x28x28x32xf32>)
                        -> tensor<1x28x28x32xf32>
    %cv = linalg.conv_2d_nhwc_hwcf
        ins(%input, %filter : tensor<1x30x30x16xf32>, tensor<3x3x16x32xf32>)
        outs(%cv_zero : tensor<1x28x28x32xf32>) -> tensor<1x28x28x32xf32>
    return %mm, %cv : tensor<8x32xf32>, tensor<1x28x28x32xf32>
  }
```

MatMul Indexing Maps

$$C(m,n) += A(m,k) * B(k,n)$$

MatMul as linalg.generic: M x N parallel, K reduction

Loop domain (m,n,k) drives A(m,k), B(k,n), C(m,n).



MatMul as linalg.generic

```
#matmul_trait = {
  indexing_maps = [
    affine_map<(m, n, k) -> (m, k)>,
    affine_map<(m, n, k) -> (k, n)>,
    affine_map<(m, n, k) -> (m, n)>
  ],
  iterator_types = ["parallel", "parallel", "reduction"]
}

func.func @matmul_generic(%A: tensor<8x16xf32>,
                        %B: tensor<16x32xf32>) -> tensor<8x32xf32> {
  %c0 = arith.constant 0.0 : f32
  %init = tensor.empty() : tensor<8x32xf32>
  %zero = linalg.fill ins(%c0 : f32)
                        outs(%init : tensor<8x32xf32>) -> tensor<8x32xf32>
  %C = linalg.generic #matmul_trait
    ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
    outs(%zero : tensor<8x32xf32>) {
    ^bb0(%a: f32, %b: f32, %c: f32):
      %p = arith.mulf %a, %b : f32
      %s = arith.addf %c, %p : f32
      linalg.yield %s : f32
    } -> tensor<8x32xf32>
  return %C : tensor<8x32xf32>
}
```

MatMul Loop Classification

Loop	역할	Iterator	NPU mapping
m	output row	parallel	PE rows / M tile
n	output column	parallel	PE cols / N tile
k	dot product	reduction	K streaming / accumulator reuse

MatMul NPU schedule questions

M/N tile 은 PE array 를 충분히 채우는가 ?

K tile 은 accumulator capacity 와 SRAM bandwidth 에 맞는가 ?

A/B tile 을 double-buffering 할 때 DMA burst 가 정렬되는가 ?

Bias/ReLU/Requant 가 matmul epilogue 로 fusion 가능한가 ?

Output layout 이 다음 op 의 preferred layout 과 호환되는가 ?

Conv2D NHWC/HWCF Indexing Maps

Output axes are parallel; filter/input-channel axes are reductions

Conv2D NHWC/HWCF as linalg.generic

Loop domain $(n, oh, ow, f, kh, kw, c)$: output axes are parallel; filter/channel axes are reductions.

Iterator classification

- parallel: $n, oh, ow, f \rightarrow$ independent output elements
- reduction: $kh, kw, c \rightarrow$ accumulate receptive field

Input NHWC

$(n, oh + kh, ow + kw, c)$

Filter HWCF

(kh, kw, c, f)

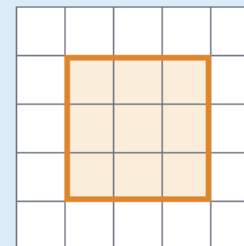
Output NHWC

(n, oh, ow, f)

NPU view

For each output tile:

- load input halo tile: $OH \times OW$ plus KH/KW border
- load filter tile: $KH \times KW \times C$ by F block
- accumulate across kh, kw, c into int32/f32 accumulators
- apply epilogue: bias, activation, requant, layout transform



3x3 receptive field

input tile footprint comes from indexing map

Conv2D NHWC/HWCF as linalg.generic

```
#conv_trait = {
  indexing_maps = [
    affine_map<(n, oh, ow, f, kh, kw, c) -> (n, oh + kh, ow + kw, c)>,
    affine_map<(n, oh, ow, f, kh, kw, c) -> (kh, kw, c, f)>,
    affine_map<(n, oh, ow, f, kh, kw, c) -> (n, oh, ow, f)>
  ],
  iterator_types = ["parallel", "parallel", "parallel", "parallel",
                    "reduction", "reduction", "reduction"]
}

func.func @conv2d_nhwc_hwcf_generic(
  %input: tensor<1x30x30x16xf32>,
  %filter: tensor<3x3x16x32xf32>) -> tensor<1x28x28x32xf32> {
  %c0 = arith.constant 0.0 : f32
  %init = tensor.empty() : tensor<1x28x28x32xf32>
  %zero = linalg.fill ins(%c0 : f32)
                      outs(%init : tensor<1x28x28x32xf32>)
                      -> tensor<1x28x28x32xf32>
  %out = linalg.generic #conv_trait
    ins(%input, %filter : tensor<1x30x30x16xf32>, tensor<3x3x16x32xf32>)
    outs(%zero : tensor<1x28x28x32xf32>) {
    ^bb0(%x: f32, %w: f32, %acc: f32):
      %p = arith.mulf %x, %w : f32
      %s = arith.addf %acc, %p : f32
      linalg.yield %s : f32
    } -> tensor<1x28x28x32xf32>
  return %out : tensor<1x28x28x32xf32>
}
```

Conv Loop Classification

Loop	의미	Iterator	NPU 관점
n	batch	parallel	outer parallelism
oh/ow	output spatial	parallel	spatial tile + halo
f	output channel	parallel	channel tile / PE mapping
kh/kw	kernel taps	reduction	filter loop
c	input channel	reduction	MAC depth / SRAM traffic

Conv tile footprint example

Output tile OH=8, OW=8, F=16, KH=3, KW=3, C=16

Input halo tile: $(8+3-1) \times (8+3-1) \times 16 = 1600$ elements

Filter tile: $3 \times 3 \times 16 \times 16 = 2304$ elements

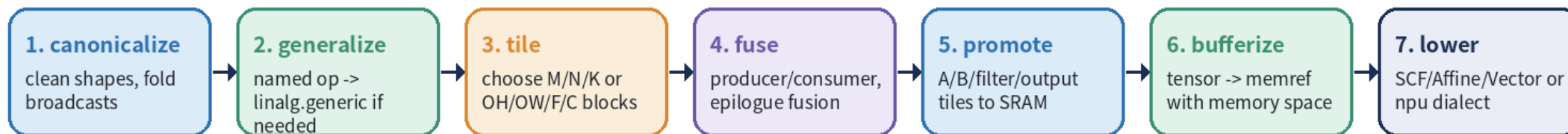
Output tile: $8 \times 8 \times 16 = 1024$ elements; accumulator byte width may be larger than input dtype

Linalg to NPU Kernel Candidate

Canonicalize -> generalize -> tile/fuse/promote -> bufferize -> lower

From Linalg-on-Tensors to an NPU Kernel Candidate

Do high-level schedule decisions before tensor value semantics are erased by bufferization.



Schedule questions for 256+ TOPS-class Edge NPU

- PE utilization**: Does the tile shape expose enough parallel M/N/F work?
- SRAM bandwidth**: Does the input/filter tile fit with double buffering?
- Accumulator pressure**: How large is the reduction tile before spill?
- Fusion legality**: Can bias/activation/requant stay in the epilogue?

Layout

When do we convert NHWC/HWCF to hardware-preferred blocked layout?

Fusion candidates at Linalg level

MatMul + Bias + Activation: best treated as epilogue fusion when legal.

Conv + Bias + ReLU/Requant: avoids output DRAM round-trip.

Elementwise producer feeding contraction may be fused if tile footprint does not explode.

Reduction producers/consumers need careful legality checks and sometimes remain separate.

Linalg Audit Checklist

질문	IR 위치	NPU risk
dynamic shape?	tensor type / tensor.dim	runtime descriptor 복잡도
iterator correct?	iterator_types	race / wrong reduction
layout conflict?	indexing_maps	late transpose/copy
fusion legal?	producer/consumer chain	DRAM traffic
SRAM fit?	maps + tile sizes	spill / poor DMA

좋은 lowering boundary

StableHLO/TOSA: semantic correctness 와 decomposition

Linalg-on-Tensors: structured schedule search, fusion, tile footprint analysis

Bufferization/MemRef: memory space, alias, lifetime, layout materialization

SCF/Vector/NPU dialect: loops, vector.contract, DMA, barrier, command buffer

Common mistakes

Named op 만 보고 access map 을 검증하지 않는다 .

Reduction loop 를 parallel 로 취급해 accumulator semantics 를 깨뜨린다 .

Conv halo 를 무시하고 SRAM footprint 를 과소평가한다 .

Tensor 단계에서 hardware-specific layout 을 너무 일찍 고정한다 .

Bufferization 이후에야 fusion 기회를 찾으려 한다 .

Exercise A - Annotation

MatMul generic IR 에서 `indexing\_maps` 의 세 map 을 각각 A/B/C 에 연결한다 .

Loop domain 과 iterator_types 를 표로 작성한다 .

region argument `%a`, `%b`, `%c` 가 어떤 scalar element 인지 표시한다 .

`linalg.yield` 가 어떤 output element update 를 의미하는지 설명한다 .

Exercise B - Write MatMul generic

M=64, N=128, K=256 인 tensor-based MatMul generic 을 작성한다 .

필수 포함 : `tensor.empty`, `linalg.fill`, `indexing_maps`, `iterator_types`, `region body`.

NPU 질문 : M/N/K tile 후보를 두 개 이상 제안하고 , SRAM/accumulator tradeoff 를 적는다 .

Exercise C - Write Conv maps

Input 1x30x30x16, Filter 3x3x16x32, Output 1x28x28x32 를 사용한다 .

Loop domain: n, oh, ow, f, kh, kw, c

Input/Filter/Output indexing map 을 작성한다 .

Output tile 8x8x16 의 input halo, filter tile, accumulator size 를 계산한다 .

Homework

MatMul + Bias + ReLU 를 Linalg-on-Tensors 로 표현하고 fusion 후보를 표시한다 .

Conv2D stride=2 case 의 input indexing map 을 작성한다 .

자신의 NPU 에서 지원할 named op 와 generic fallback policy 를 정리한다 .

다음 강의 전까지 `linalg.generic` 을 `scf.for` loop 로 수동 lower 한 pseudo IR 을 작성한다 .

Takeaways

Linalg-on-Tensors 는 NPU compiler 의 scheduling IR 로 매우 중요하다 .

`linalg.generic` 은 maps, iterators, operands, region 으로 읽으면 된다 .

MatMul 과 Conv 는 parallel/reduction loop 구분이 성능과 정확성을 모두 좌우한다 .

Tile footprint, halo, accumulator pressure, epilogue fusion 은 Linalg 단계에서 봐야 한다 .

8 강에서는 이 구조를 SCF/Affine loop 로 낮추는 방법을 다룬다 .

References

MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>

Structured Linalg Tutorial: <https://mlir.llvm.org/docs/Tutorials/transform/Ch0/>

MLIR Tensor Dialect: <https://mlir.llvm.org/docs/Dialects/TensorOps/>

MLIR Passes: <https://mlir.llvm.org/docs/Passes/>